

WORKTECC Modelling Framework – User instructions

Table of Contents

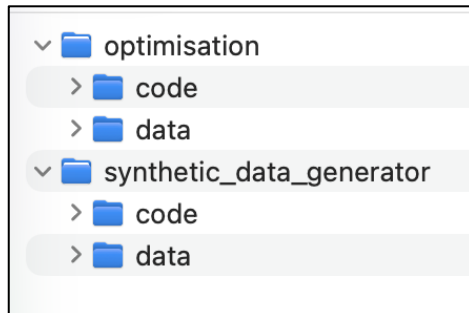
Introduction.....	2
1. Synthetic data generator	3
1.1 Input parameters to the synthetic data generator	4
1.2 Running the synthetic data generator	7
1.3 Auxiliary R functions	8
2. Optimisation	9
2.1 Input parameters for the analysis of decision problems	11
2.2 Running the decision problem analysis	13
2.3 Auxiliary R functions	16

Introduction

This analysis tool, presented in Grieco et al. 2025

(<https://doi.org/10.1101/2025.11.06.25339521>), is organised into two main folders:

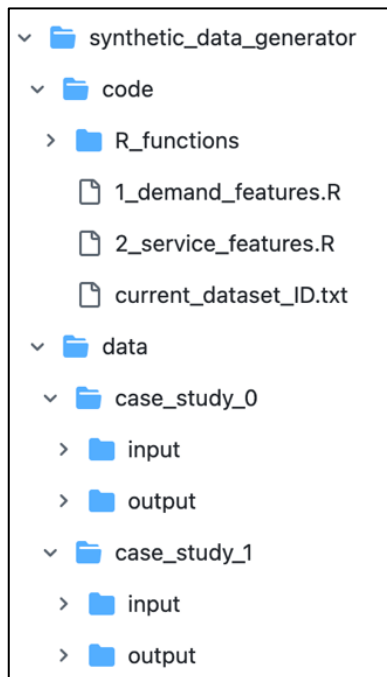
- *synthetic_data_generator*, containing the code for generating the synthetic data, the input parameters and the datasets generated;
- *optimisation*, containing the code for running the optimisation problems and heuristic algorithms as well as the results from each analysis.



The *optimisation* code reads synthetic data from the *data* folder in *synthetic_data_generator*.

1. Synthetic data generator

The synthetic data generator is structured as shown in the figure below.



The section *code* contains the R code for generating the data. The code is split into two main scripts to be run in sequence:

- **1_demand_features.R** generates the set of patients and their needs throughout the planning period;
- **2_service_features.R** generates visit types and durations, and determines the compatibility between roles and visits.

The full set of generated data is stored in a sub-folder of *data*, named after the dataset ID. For instance, in our case study, we generated two datasets: *case_study_0* and *case_study_1*. The dataset ID must be written in the file *current_dataset_ID.txt* in *code* as the tool will read this file to determine where to read parameters and where to write the output.

In order to set up the tool for generating the synthetic data, we need to create a folder in *data* named with the wished synthetic dataset ID. This folder must contain two subfolders:

- an empty folder named *output*;
- a folder named *input* that contains all the necessary parameters as R objects.

1.1 Input parameters to the synthetic data generator

Here we list the input to the synthetic data generator. For each required R object, we give a short description and provide an example taken from our case study.

- ***activity_features.rds***

Dataframe object storing, for each activity (row names), the type of activity (“DN” stands for District Nursing, “PT” stands for Physiotherapy) and its expected duration.

	type	service_time
Accident.prevention	DN	0.27
Advice.for.district.nursing	DN	0.18
Assessment	DN	0.60
Bandaging	DN	0.15
Bereavement.support	DN	0.27
Bladder.care	DN	0.18

- ***role_features.rds***

Dataframe object storing, for each role, the hourly salary and the maximum number of hours each staff member in that role can work in a day.

	role	hourly_rate	max_daily_working_hours
1	DN_L	10	5
2	PT_L	10	5
3	DN_M	14	5
4	PT_M	14	5
5	DN_H	21	5
6	PT_H	21	5

- ***activity_to_role.rds***

Dataframe object storing, for each activity (row names), whether it can (1) or cannot (0) be conducted by a staff member in each potential role.

	DN_L	DN_M	DN_H	PT_L	PT_M	PT_H
Accident.prevention	0	0	1	0	0	0
Advice.for.district.nursing	0	0	1	0	0	0
Assessment	0	0	1	0	0	0
Bandaging	1	1	0	0	0	0
Bereavement.support	0	1	1	0	0	0
Bladder.care	1	1	0	0	0	0

- ***expected_number_of_patients.rds***

Integer number specifying the expected number of patients that receive home visits at least once throughout the whole planning period. In our implementation, we use this number as parameter of the Poisson distribution to randomly generate the total number of patients.

- ***geographic_area.rds***

SpatialPolygonsDataFrame object (package “sp”) storing the shape of the territory of interest. Please refer to the documentation of the package “sp” on how to retrieve geographic representations of territories of interest. In our case study, we used a subset of wards of a borough in London. For each ward, we amended the field *@data* of this object by making sure it contains wished ward names, the population of each ward and the coordinates (x and y) of the centroid of each ward.

NAME	GSS_CODE	DISTRICT	LAGSSCODE	HECTARES	NONLD_AREA	x_centroid	y_centroid	population
Ward1	E05011244	Borough1	E09000026	181.499	0.0	546404.4	186910.3	1365
Ward2	E05011241	Borough1	E09000026	149.090	0.0	542698.0	187837.1	1866
Ward3	E05011247	Borough1	E09000026	109.143	2.3	543961.8	185414.8	1238
Ward4	E05011240	Borough1	E09000026	89.950	0.0	544305.5	186007.8	1325
Ward5	E05011248	Borough1	E09000026	186.183	0.0	545493.9	186005.7	1812
Ward6	E05011253	Borough1	E09000026	180.126	0.0	543567.9	187346.1	1375
Ward7	E05011246	Borough1	E09000026	106.244	0.0	544303.7	186753.1	656
Ward8	E05011250	Borough1	E09000026	174.392	0.0	545123.0	187979.2	1535
Ward9	E05011251	Borough1	E09000026	181.636	0.0	546166.5	187997.4	1496
Ward10	E05011237	Borough1	E09000026	155.574	0.0	547240.2	188401.6	1630

- ***overhead_visit_time.rds***

When defining visit types as combinations of activities, the duration of that visit type is given by the sum of durations of the corresponding activities plus an overhead time that accounts for the time spent by the staff member with the patient that is not considered clinical time (e.g. creating and maintaining rapport).

- ***patient_distribution.rds***

Matrix containing the expected proportion of the total number of patients belonging to a given {group, ward} pair. Note, patient groups are used here to define different patterns of needs across the planning period – once patient needs are defined, the attribute “group” is removed from the dataset.

	Ward1	Ward2	Ward3	Ward4	Ward5	Ward6	Ward7	Ward8	Ward9	Ward10
group1	0.056	0.077	0.051	0.055	0.075	0.057	0.027	0.063	0.062	0.067
group2	0.008	0.010	0.007	0.007	0.010	0.008	0.004	0.009	0.008	0.009
group3	0.010	0.013	0.009	0.009	0.013	0.010	0.005	0.011	0.010	0.011
group4	0.007	0.009	0.006	0.006	0.009	0.007	0.003	0.008	0.007	0.008
group5	0.007	0.009	0.006	0.006	0.009	0.007	0.003	0.008	0.007	0.008
group6	0.005	0.007	0.004	0.005	0.006	0.005	0.002	0.005	0.005	0.006
group7	0.002	0.003	0.002	0.002	0.003	0.002	0.001	0.002	0.002	0.002
group8	0.002	0.003	0.002	0.002	0.003	0.002	0.001	0.002	0.002	0.002

- ***patient_need_states.rds***

List containing a matrix for each patient group. The matrix defines all possible states a patient of that group might be, that is the set of activities the patient might need across a week and how many times they need each of those activities. For instance (figure below) if a patient from “group6” is in “state4” in a given week, then they need to be visited for “Rehabilitation” three times and for “Review” one time in the current week.

\$group6	state0	state1	state2	state3	state4	state5	state6	state7	state8
Rehabilitation	0	2	0	3	3	1	0	1	0
Assessment	0	0	1	0	0	0	0	0	0
Review	0	0	0	0	1	0	1	1	0
Advice.for.district.nursing	0	0	0	0	0	0	0	0	0
Ulcer.care	0	0	0	0	0	0	0	0	0
Screening	0	0	0	0	0	0	0	0	0
Physiotherapy.assessment	0	0	0	0	0	0	0	0	0
Risk.assessment	0	0	0	0	0	0	0	0	0
Wound.care	0	0	0	0	0	0	0	0	0
Daily.living	0	0	0	0	0	0	0	0	0
Advice.for.physiotherapy	0	0	0	0	0	0	0	0	0
Liaison	0	0	0	0	0	0	0	0	0
Equipment.assessment	0	0	0	0	0	0	0	0	0

- ***state_freq.rds***

List containing a vector for each patient group. The vector defines the probability that a patient of that group is in a given state (mix and frequencies of activities needed) during the first week of the planning period – patient needs for the first week are generated by sampling from these probabilities.

state0	state1	state2	state3	state4	state5
0.7354123354	0.0151074151	0.0029106029	0.0085239085	0.0001386001	0.0129591130

- ***state_switch_freq.rds***

List containing a matrix for each patient group. The matrix defines the probability that a patient of that group moves from one state (mix and frequencies of activities needed) to another state from on the next week. Patient needs are generated by sampling from these probabilities iteratively throughout the weeks in the planning period.

	state0	state1	state2	state3	state4	state5
state0	0.9342293	0.0005466792	0.006696821	0.0006898571	0	8.460512e-05
state1	0.8383838	0.0000000000	0.000000000	0.000000000	0	0.000000e+00
state2	0.7794226	0.0000000000	0.138415988	0.0014803849	0	1.480385e-03
state3	0.9491525	0.0000000000	0.016949153	0.0169491525	0	0.000000e+00
state4	0.0000000	0.0000000000	0.000000000	0.000000000	0	0.000000e+00
state5	0.1935484	0.0000000000	0.193548387	0.0000000000	0	3.225806e-02

- ***number_of_weeks.rds***

Integer number specifying the number of weeks in the whole planning period.

1.2 Running the synthetic data generator

By running the two main scripts **1_demand_features.R** and **2_service_features.R**, the folder *output* will be automatically populated with two R objects, containing the generated demand data and service data, respectively. The figure below summarises the content of these objects which will be read from the *optimisation* code to generate problem instances.

a)

Patients

patID	patient_group	basic_unit	x_coord	y_coord
pat1	group8	Ward2	542595.705	187085.3
pat2	group5	Ward10	547136.969	188860.851
pat3	group2	Ward10	546721.931	188559.066
pat4	group1	Ward5	545262.808	185382.194
pat5	group1	Ward1	547006.1	187180.855
...	...	...	...	...

Weekly patient needs

patID	group	subperiod	need	times	profile
pat3	group2	1	Blood.monitoring	1	pat_profile1
pat5	group1	1	Blood.monitoring	3	pat_profile2
pat5	group1	1	Daily.living	1	pat_profile2
pat5	group1	1	Mobility.therapy	3	pat_profile2
pat5	group1	1	Balance.therapy	3	pat_profile2
pat8	group2	1	Rehabilitation	2	pat_profile3
...	...	...	...	...	...

Estimated count of "week profiles" throughout the planning period for each ward

weekly_profile	Ward1	Ward2	Ward3	Ward4	Ward5	Ward6	Ward7	Ward8	Ward9	Ward10
pat_profile1	0	0	0	1	0	1	0	1	0	4
pat_profile2	1	0	0	0	0	1	0	0	3	1
pat_profile3	3	7	1	0	4	6	1	11	3	10
pat_profile4	1	2	3	0	1	0	0	0	5	1
pat_profile5	1	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...

b)

Activity types and expected durations

activity	type	service_time
Blood.monitoring	DN	0.13
Daily.living	DN	0.48
Mobility.therapy	PT	0.43
Balance.therapy	PT	0.25
...	...	...

Roles and salaries

role	hourly_rate
DN_L	10
PT_L	10
DN_M	14
PT_M	14
DN_H	21
PT_H	21
...	...

Potential visit types as combinations of activities

activity	visit_type1	visit_type2	visit_type3	visit_type4	visit_type5	visit_type6
Blood.monitoring	1	0	0	1	0	0
Daily.living	0	0	1	1	0	0
Mobility.therapy	0	0	0	0	1	1
Balance.therapy	0	1	0	0	0	1
...	...	...	...	...	...	...

Visit durations

visit_type	time
visit_type1	0.23
visit_type2	0.35
visit_type3	0.58
visit_type4	0.71
visit_type5	0.53
visit_type6	0.78
...	...

Visit-to-role compatibility

visit_type	DN_L	DN_M	DN_H	PT_L	PT_M	PT_H
visit_type1	1	0	0	0	0	0
visit_type2	0	0	0	1	0	0
visit_type3	1	0	0	0	0	0
visit_type4	1	0	0	0	0	0
visit_type5	0	0	0	1	0	0
visit_type6	0	0	0	1	0	0
...	...	...	...	...	...	...

Example of structured synthetic dataset formed of demand features (a) and service features (b).

The output R objects will both contain the dataset ID for reference in the optimisation code. For instance, for our case study with ID *case_study_0*, the output objects are named ***demand_data_case_study_0.rds*** and ***service_data_case_study_0.rds***.

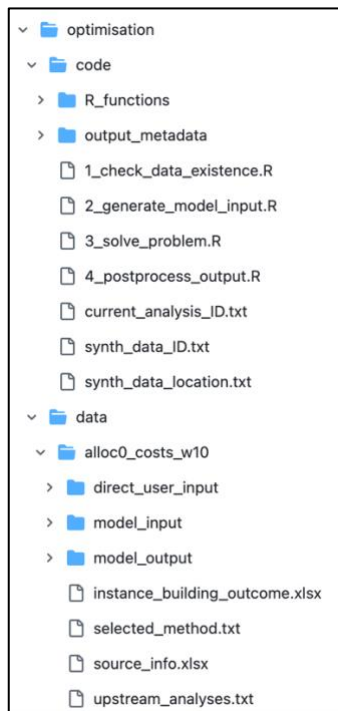
1.3 Auxiliary R functions

The section *code* also contains a folder named *R_functions* where all the auxiliary functions for generating the data are stored. In particular:

- **generatePatients.R** randomly generates locations for a given number of patients across the geographic area provided as input, assigning them to patient groups based on the specified distribution;
- **generatePatientNeeds.R** generates the set of activities (with frequencies) needed by each patient for each week in the planning period, based on initial probabilities for a patient in a given group to have a given set of needs and on probabilities to switch from one set of needs to another from week to week;
- **generateServiceData.R** takes as input a list of activities (with durations) and a compatibility matrix between activities and potential roles, and generates the set of potential visit types (as combinations of activities), visit durations and compatibility between roles and visit types.

2. Optimisation

The optimisation part is structured as shown in the figure below.



The section *code* contains the R code for analysing a decision problem. The code is split into four main scripts to be run in sequence:

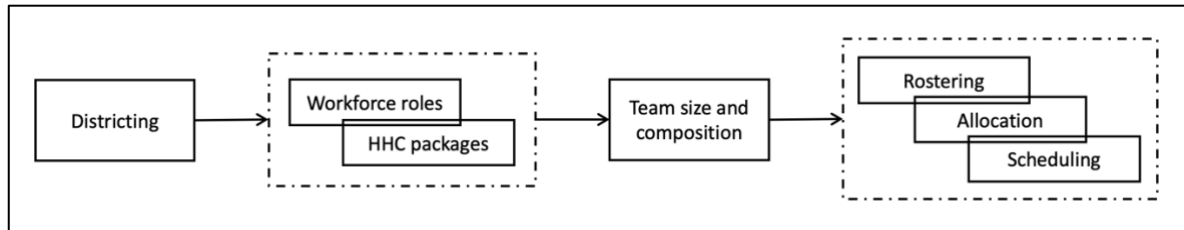
- **1_check_data_existence.R** checks whether all parameters are available either from synthetic data or as direct user input;
- **2_generate_model_input.R** generates the problem instance based on the selected solution method;
- **3_solve_problem.R** generates a solution to the problem using the selected optimisation or heuristic method;
- **4_postprocess_output.R** computes a vector of performance metrics from the obtained solution.

All information about the decision problem analysed is stored in a sub-folder of *data*, named after the analysis ID. For instance, the figure above shows such sub-folder for the analysis with ID *alloc0_costs_w10* (see sections below for more details on that). The analysis ID must be written in the file *current_analysis_ID.txt* in *code* as the tool will read this file to determine where to read parameters and where to write the output.

The reference synthetic dataset ID (e.g. *case_study_0*) must be written in the file *synth_data_ID.txt* in *code* as the tool will look for the corresponding folder in the synthetic data generator section to check whether all relevant data are available and to retrieve them. We also have a file named *synth_data_location.txt* that must contain the

relative path from the folder *code* in *optimisation* to the folder *data* in *synthetic_data_generator*. For instance, in our case, this path is:
`../../synthetic_data_generator/data`.

In our case study, we implemented the framework to analyse a sequence of decision problems as depicted in the following figure (cf. Grieco et al 2025).



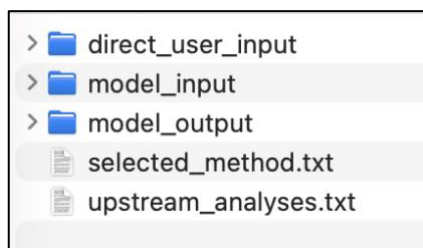
Decision problems have to be analysed according to the specified hierarchy. Dashed boxes denote sets of decisions that are made simultaneously, that is under the same analysis ID (e.g. one optimisation problem addressing “Workforce roles” and “HHC packages” decisions at the same time).

In our case study, we run the above sequence of decisions for both synthetic datasets *case_study_0* and *case_study_1*. We started by analysing the decision problem “Districting”, creating the corresponding analyses with IDs *distr0* and *distr1*, respectively. We then focused on only one of the districts generated with this analysis (“distr1”) and we analysed the following decision problem in the hierarchy (“Workforce roles & HHC packages”) by creating the analyses with IDs *pack0* and *pack1*. The following analyses were *team0* and *team1* that gave us a solution for “Team size and composition”. The last step consisted in analysing the decision problem “Rostering & Allocation & Scheduling”. Differently from the previous analyses focusing on the whole planning period of 20 weeks, we analysed this last decision problem by focusing on specific weeks (due to how the formulation is structured). Furthermore, we tested two different optimisation criteria at this step: minimisation of number of staff-patient pairs and minimisation of monetary costs. Therefore, we defined eight analyses: *alloc0_pairs_w2*, *alloc0_pairs_w10*, *alloc0_costs_w2*, *alloc0_costs_w10*, *alloc1_pairs_w2*, *alloc1_pairs_w10*, *alloc1_costs_w2*, *alloc1_costs_w10*.

2.1 Input parameters for the analysis of decision problems

In order to set up the tool for analysing a decision problem, we need to create a folder in *data* with name corresponding to the wished analysis ID. This folder must contain the following files and sub-folders (see figure below):

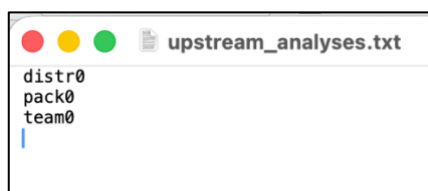
- a text file named *selected_method.txt* that must contain the ID of the solution algorithm to be used;
- a text file named *upstream_analyses.txt* that must contain the IDs of existing upstream analyses from which we want to base the instance of the current decision problem;
- a folder named *direct_user_input* to be populated with parameters that will not be read from the synthetic dataset;
- an empty folder named *model_input*;
- an empty folder named *model_output*.



As an example, we show here how we set up the analysis with ID *alloc0_costs_w10*.

The selected method ID must correspond to the name of the solution algorithm chosen for the problem. For the “Rostering & Allocation & Scheduling” problem, we named this algorithm *RosteringANDAllocationANDScheduling1*, therefore this is the label to be written in the file *selected_method.txt*. Note, the framework can be set up with different algorithms to solve the same decision problem.

The file *upstream_analyses.txt* must contain the list of analyses that are conducted upstream of the current analysis, so the tool is able to look for relevant parameters in the folders of those analyses. For *alloc0_costs_w10*, the IDs of the upstream analyses are: *distr0*, *pack0*, *team0* (see figure below).



The folders *model_input* and *model_output* should be left empty – they will be automatically populated by executing the main R code.

The folder *direct_user_input* must contain a set of R objects encoding model parameters that are not accounted for by the synthetic data or that are necessary to identify relevant subsets of the synthetic data. The list of such R objects depends on the specific analysis (ideally, they should be independent of the method use for that decision problem).

```
> districts_of_interest
[1] "distr1"
> avg_travel_speed
[1] 15
> mult_eucl_to_real
[1] 1.4
> budget_limit
[1] 1e+09
> max_unique_pairs
[1] 1e+09
> min_prop_visits
[1] 1
> subperiods
[1] 1 2 3 4 5 6 7
> ref_time_period
[1] 10
> obj_weights
weight_costs weight_pairs
           1           0
```

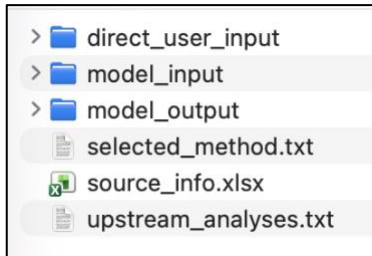
For instance, the figure above shows the set of R objects required for *alloc0_costs_w10*:

- the objects “budget_limit”, “max_unique_pairs”, “min_prop_visits”, “subperiods” and “obj_weights” are directly used to parameterise the programming formulation;
- the objects “districts_of_interest” and “ref_time_period” are used to select relevant data from the synthetic dataset and/or from upstream analysis results;
- the objects “avg_travel_speed” and “mult_eucl_to_real” are parameters used to transform the retrieved synthetic data and/or upstream analysis results into model parameters (e.g. determining the travel time between two patient locations defined as geographic coordinates).

For direct user inputs of other decision problems, please refer to the R code provided and to Grieco et al. 2025.

2.2 Running the decision problem analysis

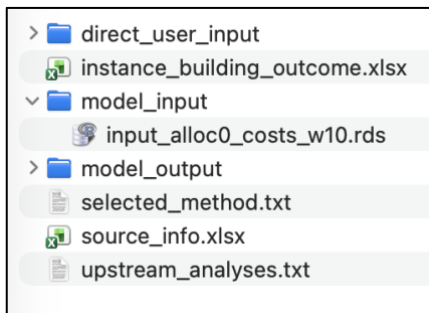
The first script to run is **1_check_data_existence.R**. This script tries to locate all direct user input, synthetic data and upstream output that are necessary to create an instance for the current decision problem. A spreadsheet named *source_info.xlsx* is created to inform the user about the outcome of the check.



	A	B	C
5	association between staff and roles	upstream	Association between staff and roles identified from upstream analysis team0
6	salary	synthetic	Salary per staff identified from synthetic data
7	sub periods	user	Provide file subperiods.rds in direct_user_input folder of alloc0_costs_w10
8	staff availability	upstream	Staff availability identified from upstream analysis team0
9	list of visits	upstream and synthetic	List of visits identified from synthetic datasets and upstream analysis pack0
10	association between visits and patients	upstream and synthetic	Association between visits and patients identified from synthetic datasets and upstream analysis pack0
11	association between staff members and visits they can conduct	upstream and synthetic	Association between staff members and visits they can conduct identified from synthetic datasets and upstream analysis pack0
12	districts	upstream	Districts identified from upstream analysis distr0. Provide file districts_of_interest.rds in direct_user_input folder of alloc0_costs_w10
13	service times	upstream and synthetic	Service times identified from synthetic datasets and upstream analysis pack0
14	within district travel distance	upstream and synthetic	Within district travel time identified from synthetic datasets and upstream analysis pack0
15	maximum workload per staff member	synthetic	Maximum workload per staff member identified from synthetic data
16	average travel speed	user	Provide file avg_travel_speed.rds in direct_user_input folder of alloc0_costs_w10
17	minimum proportion of visits to be conducted	user	Provide file min_prop_visits.rds in direct_user_input folder of alloc0_costs_w10
18	maximum number of unique staff patient pairs	user	Provide file max_unique_pairs.rds in direct_user_input folder of alloc0_costs_w10
19	max budget	user	Provide file budget_limit.rds in direct_user_input folder of alloc0_costs_w10
20	weights of objective function components	user	Provide file obj_weights.rds in direct_user_input folder of alloc0_costs_w10
21	euclidean to real distance factor	user	Provide file mult_eucl_to_real.rds in direct_user_input folder of alloc0_costs_w10
22	reference time period	user	Provide file ref_time_period.rds in direct_user_input folder of alloc0_costs_w10

The spreadsheet contains a list of all the data needed for creating the model instance. Errors and warnings will appear in this spreadsheet which will need to be fixed to ensure a correct running of the next script.

The script **2_generate_model_input.R** accesses the data identified above and processes them to create the model instance. Similarly to the previous step, a spreadsheet named *instance_building_outcome.xlsx* is created reporting which data were correctly processed. At the same time, an R object called *input_alloc0_costs_w10.rds* is created in the *model_input* subfolder containing all the processed data, ready to be either encoded into a programming formulation or used in a heuristic algorithm.



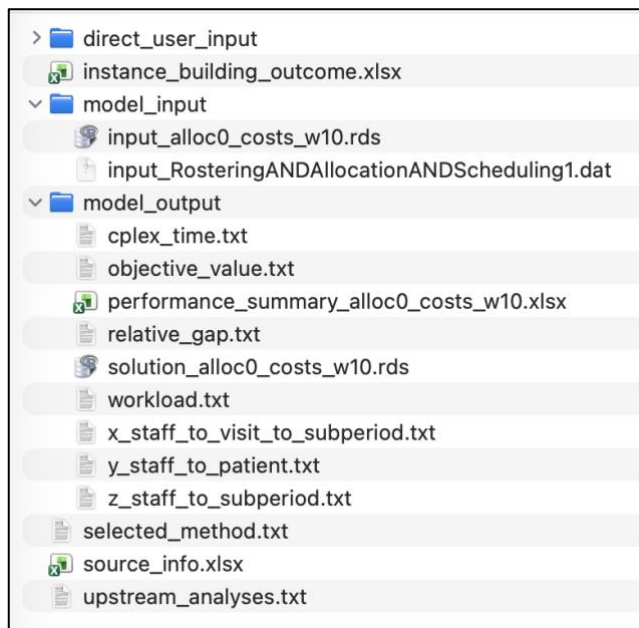
	A	B	C	D
1	data_element	source_type	description	building_outcome
2	list of patients	upstream and synthetic	List of patients identified from synthetic datasets and upstream analysis pack0	List of patients identified from upstream analysis and synthetic data
3	list of staff members	upstream	List of staff members identified from upstream analysis team0	List of staff members identified from upstream analysis
4	staff roles	upstream	Staff roles identified from upstream analysis team0	Staff roles identified from upstream analysis
5	association between staff and roles	upstream	Association between staff and roles identified from upstream analysis team0	Association between staff and roles identified from upstream analysis
6	salary	synthetic	Salary per staff identified from synthetic data	Salary identified from synthetic data
7	sub periods	user	Provide the subperiods.rds in direct_user_input folder of alloc0_costs_w10	Sub periods read from subperiods.rds as direct user input
8	staff availability	upstream	Staff availability identified from upstream analysis team0	Staff availability identified from upstream analysis
9	list of visits	upstream and synthetic	List of visits identified from synthetic datasets and upstream analysis pack0	List of visits identified from upstream analysis and synthetic data
10	association between visits and patients	upstream and synthetic	Association between visits and patients identified from synthetic datasets and upstream analysis pack0	Association between visits and patients identified from upstream analysis and synthetic data
11	association between staff members and visits they can conduct	upstream and synthetic	Association between staff members and visits they can conduct identified from synthetic datasets and upstream analysis pack0	Association between staff members and visits they can conduct identified from upstream analysis and synthetic data
12	districts	upstream	Districts identified from upstream analysis team0. Provide the districts_of_interest.rds in direct_user_input folder of alloc0_costs_w10	Districts identified from upstream analysis using information provided in districts_of_interest.rds as direct user input
13	service times	upstream and synthetic	Service times identified from synthetic datasets and upstream analysis pack0	Service times identified from upstream analysis and synthetic data
14	within district travel distance	upstream and synthetic	Within district travel time identified from synthetic datasets and upstream analysis pack0	Within district travel distance identified from upstream analysis and synthetic data
15	maximum workload per staff member	synthetic	Maximum workload per staff member identified from synthetic data	Maximum workload per staff member identified from synthetic data
16	average travel speed	user	Provide file avg_travel_speed.rds in direct_user_input folder of alloc0_costs_w10	Average travel speed read from avg_travel_speed.rds as direct user input
17	minimum proportion of visits to be conducted	user	Provide file min_prop_visits.rds in direct_user_input folder of alloc0_costs_w10	Minimum proportion of visits to be conducted read from min_prop_visits.rds as direct user input
18	maximum number of unique staff patient pairs	user	Provide file max_unique_pairs.rds in direct_user_input folder of alloc0_costs_w10	Maximum number of unique staff patient pairs read from max_unique_pairs.rds as direct user input
19	max budget	user	Provide the budget_limit.rds in direct_user_input folder of alloc0_costs_w10	Max budget read from budget_limit.rds as direct user input
20	weights of objective function components	user	Provide file obj_weights.rds in direct_user_input folder of alloc0_costs_w10	Weights of objective function components read from obj_weights.rds as direct user input
21	eucledian to real distance factor	user	Provide file mult_eucl_to_real.rds in direct_user_input folder of alloc0_costs_w10	Eucledian to real distance factor read from mult_eucl_to_real.rds as direct user input
22	reference time period	user	Provide the ref_time_period.rds in direct_user_input folder of alloc0_costs_w10	Reference time period read from ref_time_period.rds as direct user input

The script **3_solve_problem.R** generates the solution to the decision problem. In our case study, one of the four methods used (namely, for “Team size and composition”) is a heuristic algorithm provided as an R script. This script is directly run using the input object created at the previous step. The methods chosen for the remaining three decision problems are based on mathematical programming – the tool calls CPLEX to find an optimal solution to the problem (therefore a CPLEX installation is required to run this tool). Before calling the solver, the tool translates the input R object into a *.dat* file to be used as input to CPLEX. This file is added to the sub-folder *model_input*. After a solution is found, *.txt* files are saved in the sub-folder *model_output* containing the optimal values of all decision variables as well as some technical information such as the “CPLEX time” and the relative optimality gap.

>	direct_user_input
	instance_building_outcome.xlsx
▼	model_input
	input_alloc0_costs_w10.rds
	input_RosteringANDAllocationANDScheduling1.dat
▼	model_output
	cplex_time.txt
	objective_value.txt
	relative_gap.txt
	workload.txt
	x_staff_to_visit_to_subperiod.txt
	y_staff_to_patient.txt
	z_staff_to_subperiod.txt
	selected_method.txt
	source_info.xlsx
	upstream_analyses.txt

The final script **4_postprocess_output.R** reads the optimal values of all decision variables and it produces two additional files that are saved in the sub-folder *model_output* as well:

- *solution_alloc0_costs_w10.rds* is an R object containing the same data as *input_alloc0_costs_w10.rds*, plus additional objects storing the optimal values of the decision variables;
- *performance_summary_alloc0_costs_w10.xlsx* is a spreadsheet containing the value of a vector of performance metrics specifically defined for each decision problem – this is the main output based on which we compared the different scenarios described in Grieco et al. 2025.



The sub-folder *output_metadata* in *code* contains a mapping between the dimension of all output objects produced by CPLEX and the dimension names of the corresponding R object. Note, the dimension names must correspond to the original ones defined in *input_alloc0_costs_w10.rds*. A spreadsheet with such mapping needs to be produced for each programming formulation used in the analysis (cf. figure below for “Rostering & Allocation & Scheduling”).

	A	B
1	output_object	dimensions
2	x_staff_to_visit_to_subperiod	staff_members,visits,subperiods
3	y_staff_to_patient	staff_members,patients
4	z_staff_to_subperiod	staff_members,subperiods
5	workload	staff_members
6	relative_gap	
7	cplex_time	
8	objective_value	

2.3 Auxiliary R functions

The section *code* also contains a folder named *R_functions* where all the auxiliary functions are stored. These are organised into the following folders:

- *cplex_interface* contains functions that map R objects into CPLEX arrays and viceversa;
- *data_checks* contains functions (specifically for each method) that are called by the script **1_check_data_existence.R** to determine whether all necessary data are available;
- *generate_input* contains functions (specifically for each method) that are called by the script **2_generate_model_input.R** to access relevant data and generate model instances;
- *solvers* contains R functions (specifically for each method) that are called by the script **3_solve_problem.R** to compute solutions to the decision problems, either by directly executing a heuristic algorithm or by creating a CPLEX instance and calling the solver – this folder also contains a sub-folder storing all CPLEX formulations as *.mod* files;
- *reading_output* contains R functions that read CPLEX output and translate it into R objects, as well as R functions (specifically for each method) that are called by the script **4_postprocess_output.R** to compute a vector of performance metrics for the found solution – a sub-folder is defined for each method (and named after the method ID) containing functions to compute every performance metric.